HW06-AI — API Testing Main Report

1. Student and Assignment Information

Field	Value
Student ID	23127261
Student name	Vương Ngũ Tín Thành
Class	23KTPM2
Assignment	HW06-AI — API Testing
System under test	EShop
SUT repository	https://github.com/ttbhanh/eshop-sut
Submission repository	https://github.com/vntthanh/SoftwareTesting-HW06
Report date	2026-08-18
Last updated	2026-08-23

2. Selected APIs

Pool	Feature	Selected API	Main Testing Focus
A	FR-03 – Forgot Password / Password Reset	`POST /api/reset-password`	Input validation, reset-token behavior, password rules, security
B	FR-09 – Discount Coupons	`POST /api/apply-coupon`	Coupon eligibility, amount boundaries, user constraints, calculation
C	FR-18 – Admin Order Management	`PUT /api/admin/orders/:id/status`	Authorization, order-state transitions, invalid transitions

Execution identity evidence: the Postman console in `bug_fr03-unsupported-content-type-server-error.png` shows the collection pre-request log `X-Student-Id: 23127261` for a real request to the local SUT. The same collection-level injection pattern is used by all three reviewed collections, and the Newman artifacts retain the corresponding runtime assertions or request headers.

The completed two-commit GitHub Actions evidence is documented in `CI_CD_Report.md`. SUT commit `da76e9a` runs one existing reviewed case that passes without changing its assertion; its child commit `b95838e` expands CI to all 241 reviewed cases and preserves the genuine failing assertions. Both run links, uploaded-artifact names, timestamps, totals, and screenshots are included in the CI/CD report.

3. Agent Skills

3.1. Agent Skills Overall

Component	Type	Responsibility
API Test Generator	Coordinator	Coordinates the complete API test-generation process and combines results
Domain Test Generator	Specialized generator	Performs equivalence partitioning and Boundary Value Analysis
State Transition Test Generator	Specialized generator	Models states and valid/invalid transitions when applicable

Security Test Generator	Specialized generator	Derives security tests from applicable SEC-01–SEC-07 requirements
Contract Test Generator	Specialized generator	Validates request and response contracts/schema
AI Audit	Reused supporting skill	Records AI interactions and file modifications into the AI Audit Report

The first five components form the **AI-driven API Test Generator architecture** designed for HW06.

The **AI Audit** skill (reused from homework `HW05-AI`) operates alongside this architecture. It records the AI-assisted workflow so that the generation, review, correction, and implementation activities remain auditable.

3.2. Skills Design

The design artifacts for the **API Test Generator** and its four specialized generators are stored in the `skills-design/` folder. Each generator has its own subfolder containing its design specification and corresponding diagram.

The **AI Audit** skill is not included in this folder because it is a reusable skill carried over from HW05.

3.3. Agent Skill Demonstrations

Agent skill	Demonstration
API Test Generator	Stage 1 • Stage 2
Postman Test Generator	https://youtu.be/cua5521dNAA
Newman Result Analyzer	https://youtu.be/clgm8yc12GM
Append Bug (HW02)	https://youtu.be/G0DPfJ1dkqE
AI Audit (HW02)	https://youtu.be/1gSqi7MfGV

4. Postman Features Used

The project used the following Postman and Newman features:

Feature	How it was used	Evidence
Postman Collection v2.1	Saved the three API test suites as JSON collection files.	Pool A, Pool B, and Pool C
Collection variables	Stored values such as base URLs, student ID, tokens, order IDs, and OTP values. Requests used these values with `{{variable}}`.	Variable sections in the three collections
Pre-request scripts	Added the `X-Student-Id: 23127261` header before requests. Pool B and Pool C also reset test data before testing.	Collection-level `prerequest` scripts
Test scripts and assertions	Checked status codes, headers, response bodies, authentication, and saved state with `pm.test` and `pm.expect`.	Request-level `test` scripts
Request chaining	Used `pm.sendRequest` to prepare OTP data and check saved order data after a request.	Pool A setup scripts and Pool C state-checking scripts
Multi-request test flows	Grouped several requests under one test ID for retry, replay, authorization, and order state tests.	Test folders in Pool A and Pool C
Bearer token testing	Tested Admin and non-Admin tokens, as well as missing, malformed, forged, expired, and invalid tokens.	Authorization headers and variables in Pool B and Pool C
Newman command-line execution	Ran the Postman collections outside the Postman application.	Pool A report, Pool B report, and Pool C report
JSON and HTML reports	Created JSON files for detailed results and HTML files for easier review.	Newman reports and CI/CD report

Pool A: Forgot Password

A.1. Introduction

Pool A covers **FR-03 – Forgot Password / Password Reset**. This report selects `POST /api/reset-password`, which resets a user's password using an email address, reset token, and new password.

A.2. Contractual Testing Analysis

The selected endpoint is documented as `POST /api/reset-password` with a JSON body containing three top-level fields:

Field	Documented representation	Reviewed constraint
`email`	JSON string	Required for account identification and OTP-to-email binding
`resetToken`	JSON string	Required; represents the six-digit OTP generated during step 1
`newPassword`	JSON string	Required; must satisfy the FR-01 password-strength rules incorporated by FR-03

The reviewed contract model contains rules `CR-001` – `CR-012`. It covers the method and path, JSON-object representation, documented member set, string representations, OTP/email binding, password strength, OTP expiry, one-time use, the combined valid request, and the absence of a documented response schema.

The API specification does not define response statuses or bodies for this endpoint. The reviewed test model therefore labels `Content-Type: application/json`, `200 OK` for a normal successful reset, and `400 Bad Request` for invalid request data as external HTTP testing assumptions. It does not invent response messages, response fields, validation precedence, or parser behavior beyond the reviewed assumptions.

FR-03 requires a confirmation-password value at workflow/UI level, but the endpoint body does not document such a field. Consequently, `CR-008` remains unresolved for direct API testing: no `confirmPassword`-style property was invented. The complete analysis is stored in `review/pool-a/reports/contract-report.md`.

A.3. Domain Testing Analysis

Domain analysis uses the documented example as a value baseline and adds the required workflow state: a registered email, a six-digit OTP issued for that same email, an unexpired and unused OTP, and a conforming new password. Tests vary one input factor at a time where possible.

The reviewed model defines 26 equivalence partitions (`DP-001` – `DP-026`) across the JSON container and the three body fields. Important partitions include:

- valid and malformed/non-object JSON containers;
- missing, `null`, and wrong-type values for all three required fields;
- registered, unregistered, malformed, empty, and OTP-mismatched email values;
- OTP values below, at, and above six digits; non-decimal, wrong-type, omitted, and unissued values;
- passwords that meet all rules or separately omit uppercase, lowercase, digit, allowed special character, or minimum length.

Six boundary models (`DB-001` – `DB-006`) cover OTP length, password length, and the minimum counts of uppercase, lowercase, digit, and allowed-special characters. No unsupported maximum lengths, email boundaries, or concrete OTP lifetime were inferred.

Additional JSON properties remain exploratory because the specification defines no accept/reject oracle (`API-030`). Conversely, `Aa1!bbbb#` is valid (`API-059`) because it satisfies every explicit password rule through `A`, lowercase letters, `1`, and the allowed `!`; no requirement prohibits the additional `#`. The complete analysis is stored in `review/pool-a/reports/domain-report.md`.

A.4. State Transition Testing Analysis

State-transition testing is applicable because FR-03 and SEC-07 define an OTP lifecycle. The reviewed model contains three normalized states:

State	Meaning
`ST-01`	OTP is usable, unexpired, unused, and bound to the requesting email
`ST-02`	OTP has passed the real SUT expiry point and is no longer usable
`ST-03`	OTP was used successfully and invalidated

Five transitions/guard failures (`TR-001` – `TR-005`) cover successful reset and OTP invalidation, cross-email OTP use, expired OTP use, replay after successful use, and weak-password rejection. The reviewed `TR-005` decision preserves the usable OTP and leaves the old password unchanged when password validation fails.

Expiry tests do not assume a lifetime, clock, boundary, or grace period. They execute only when the real SUT expiry point can be configured or objectively observed; otherwise, the case is recorded as `BLOCKED / NOT EXECUTABLE` , not as a product failure. The complete analysis is stored in `review/pool-a/reports/state-report.md` .

A.5. Security Testing Analysis

All supplied requirements `SEC-01` – `SEC-07` were evaluated for this endpoint:

Requirement	Applicability and treatment
`SEC-01`	Applicable: the reset password must not be stored in plaintext. Verification is explicitly white-box/storage-level because Postman alone cannot prove persistence behavior.
`SEC-02`	Not applicable under the reviewed recovery model: the OTP is the recovery credential and no JWT is required for this unauthenticated endpoint.
`SEC-03`	Not applicable: this is not an Admin API.
`SEC-04`	No direct API scenario: the sources define no response-reflection or UI-rendering sink for the submitted values.
`SEC-05`	Conditionally applicable: inert SQL/query metacharacters must not bypass reset controls, affect another account, reveal database information, or cause unexpected failure. Full parameterization proof may require implementation inspection.
`SEC-06`	Not applicable: this is not a profile-update API and has no documented `role` input.
`SEC-07`	Directly applicable: the OTP is six digits, email-bound, expiring, and invalidated after successful use.

The security model contains nine scenarios (`SS-001` – `SS-009`). In addition to SEC-01, SEC-05, and SEC-07 coverage, the reviewed model includes two external best-practice scenarios: automated-guessing resistance and account-enumeration resistance. The guessing test runs only when an authoritative configured abuse-control limit is known and never invents a threshold. The enumeration test compares the same `400` status, semantic JSON body (or exact non-JSON body), content type, redirect behavior, password/token/account-metadata side effects, and treats timing as informational unless an approved tolerance exists.

The complete analysis is stored in `review/pool-a/reports/security-report.md` .

A.6. AI-Generated Test Cases

The final reviewed test cases for this pool are stored in `test-cases/a-forgot-password.csv` . The audited AI-generated subset is stored in `review/pool-a/candidate-api-tests.csv` .

The test cases were derived from the reviewed analyses in Sections A.2–A.5.

Testing Type	Number of Test Cases
Contractual Testing	26

Domain Testing	36
State Transition Testing	5
Security Testing	9
Total	**76**

The suite uses stable IDs `API-001` – `API-082` and exactly nine traceability fields per record. Every case targets `POST /api/reset-password`. The 76 specialist-generated cases retain their provisional specialist IDs in `Assumptions / Notes`; `API-077` – `API-082` are the six human-authored additions documented in Section A.7. The audited candidate artifact adds a `Verdict` and an `Audit Reason` to every AI-generated row; all retained cases are `VALID (after revise)` with case-level reasoning. Validation found no missing fields, duplicate IDs, endpoint/category mismatches, or unexpected logical cases. Coverage includes `CR-001` – `CR-007`, `CR-009` – `CR-012`, `DP-001` – `DP-026`, `DB-001` – `DB-006`, `TR-001` – `TR-005`, and `SS-001` – `SS-009`; `CR-008` is explicitly unresolved because confirmation-password transport is not documented.

Eight specialist-generated cases were revised after review: `API-025`, `API-030`, `API-059`, `API-065`, `API-068`, `API-072`, `API-075`, and `API-076`. These revisions clarified exploratory behavior, valid additional password characters, observable/configurable expiry preconditions, white-box storage verification, configured rate limits, and account-enumeration comparison rules. Six human-authored cases were then added as `API-077` – `API-082`. Revalidation preserved all 82 IDs and coverage.

Pool A execution fixtures are seeded only after the SUT starts, because the SUT recreates its SQLite schema on startup. The idempotent fixture script opens `backend/database.sqlite` directly (without importing SUT `database.js`), creates dedicated deterministic accounts and exact TEXT OTPs, and never calls `/api/forgot-password`. Expiry cases `API-025`, `API-065`, and `API-072` remain blocked because the actual SUT has no expiry state/check; `API-075` remains blocked because it has no rate limiter or authoritative threshold; and `API-077` remains manual because sequential Postman/Newman cannot establish its concurrency barrier.

A.7. Human Cases

ID	Category	Test case	Expected result	Notes
API-077	STATE	Send **two concurrent** reset requests with the same email/OTP but different valid new passwords.	Exactly **one succeeds**; the other is rejected. The OTP must authorize only one reset.	Previous AI tested sequential replay but explicitly did not test concurrency.
API-078	CONTRACT	Send otherwise valid JSON text using `Content-Type: text/plain`. Then retry normally with the same OTP using `application/json`.	The first response is any `4xx` client-error status and does not consume the OTP; statuses outside `400` – `499` are not accepted. `415 Unsupported Media Type` is the human/external HTTP expectation, but any safe `4xx` rejection, including `400 Bad Request`, is acceptable when the OTP remains intact. The second request succeeds.	The specification documents a JSON body but no response status for this endpoint; both the accepted `4xx` class and preferred `415` are human/external HTTP expectations, not specification requirements.
API-079	CONTRACT	Send `Content-Type: application/json` with an **empty HTTP body**. Then retry with a valid request using the same OTP.	Empty request returns `400`, not `5xx`; OTP remains usable and the valid retry succeeds.	Previous tests omit individual fields but do not exercise the completely missing body before destructuring.

API-080	DOMAIN	With a valid email/OTP, send `newPassword` as an object: `{"value":"Aa1!aaaa"}`. Then retry with a valid string using the same OTP.	First request returns `400`, not `5xx`, and does not consume the OTP. Second request succeeds.	Previous AI used a number as its representative non-string value.
API-081	SECURITY	Account A: reset using a valid OTP and a password containing SQL syntax while still meeting password rules, e.g. `Aa1!';DROP TABLE users;--`. Then use a valid OTP for Account B in another reset request.	A's password is treated only as data; its reset succeeds normally. B's reset also succeeds, proving the injected password did not damage the users data path.	Existing SQL-injection testing attacks `email` only. The `newPassword` field could also be attacked.
API-082	SECURITY	Send a valid reset request with an invalid/garbage `Authorization: Bearer ...` header.	Reset still succeeds with `200`; an invalid JWT must not affect this public recovery endpoint.	JWT is not applicable but we should test the case where a bad JWT header is actually present.

The AI missed these cases mainly because the initial generation focused on test cases that could be derived directly from the API specification. Concurrency and server-side persistence require deeper execution and state reasoning, so cases such as simultaneous OTP reuse were not identified.

Alternative JSON input representations were also under-sampled because the domain analysis focused on representative equivalence partitions rather than different structural representations of the same input. In addition, the initial security analysis used representative SQL injection cases but did not systematically apply them to every client-controlled field.

These gaps show that specification-based AI generation can provide wide coverage, but human review is necessary to identify execution-dependent, state-dependent, and less obvious security cases.

A.8. Newman Execution Analysis

Run summary

Metric	Normalized result
Collection	`Pool A - Reset Password Reviewed Tests`
Execution window	2026-08-22 12:42:53.158 to 12:43:00.739 (GMT+7)
Duration	7.581 seconds
Intended collection leaf requests	88
Total requests	85 (Newman `run.stats`; 82 executed collection leaf requests plus 3 pre-request helper calls)
Logical test cases	82 total: 77 with an observed execution and 5 explicitly blocked before request dispatch
Assertions	82 total: 59 passed, 23 failed, 0 pending/skipped
Request errors	0
Pre-request script errors	0
Test script errors	0
Logical cases with failed automated assertions	22
Logical execution statuses	`PASS`: 55; `FAIL_ASSERTION`: 22; `FAIL_MANUAL`: 1; `BLOCKED_NOT_EXECUTED`: 4; `REQUEST_ERROR`: 0; `RUNTIME_ERROR`: 0; `NOT_EXECUTED`: 0

Manual oracle requirements	23 logical cases explicitly retain a manual, white-box, state/side-effect, concurrency, or data-driven check

HTTP 4xx/5xx responses are classified only through their automated assertions; they are not treated as failures by status code alone. Failed assertions below are execution evidence, not defect diagnoses.

Logical test-case outcomes

Test ID	Execution Status	Request / Flow Step	HTTP Status	Assertion Result	Failure / Error Message
API-001	`PASS`	Verify the documented method, path, JSON-object member representations, and reviewed successful-reset response oracle with a fully valid reset request.	200 OK	PASS — API-001 - reviewed status is 200	N/A
API-002	`PASS`	Verify that malformed JSON is treated as invalid request data under the reviewed validation model.	400 Bad Request	PASS — API-002 - reviewed status is 400	N/A
API-003	`PASS`	Verify that a syntactically valid non-object JSON body is treated as invalid request data.	400 Bad Request	PASS — API-003 - reviewed status is 400	N/A
API-004	`PASS`	Verify that omission of reviewed-required member email is rejected.	400 Bad Request	PASS — API-004 - reviewed status is 400	N/A
API-005	`PASS`	Verify that null is rejected for reviewed-required string member email.	400 Bad Request	PASS — API-005 - reviewed status is 400	N/A
API-006	`PASS`	Verify that a non-string JSON value is rejected for email without coercion.	400 Bad Request	PASS — API-006 - reviewed status is 400	N/A
API-007	`PASS`	Verify that omission of reviewed-required member resetToken is rejected.	400 Bad Request	PASS — API-007 - reviewed status is 400	N/A
API-008	`PASS`	Verify that null is rejected for reviewed-required string member resetToken.	400 Bad Request	PASS — API-008 - reviewed status is 400	N/A

API-009	`FAIL_ASSERTION`	Verify that a valid issued resetToken value is rejected when represented as a JSON number rather than a string.	200 OK	FAIL — API-009 - reviewed status is 400	expected response to have status code 400 but got 200
API-010	`PASS`	Verify rejection of a five-digit token that does not meet the reviewed exact six-decimal-digit workflow shape.	400 Bad Request	PASS — API-010 - reviewed status is 400	N/A
API-011	`PASS`	Verify rejection of a seven-digit token under the reviewed exact six-decimal-digit reset workflow rule.	400 Bad Request	PASS — API-011 - reviewed status is 400	N/A
API-012	`PASS`	Verify rejection of a six-character token containing a non-decimal character.	400 Bad Request	PASS — API-012 - reviewed status is 400	N/A
API-013	`PASS`	Verify that an issued six-digit OTP beginning with zero remains valid through the documented string representation.	200 OK	PASS — API-013 - reviewed status is 200	N/A
API-014	`PASS`	Verify rejection of a well-shaped six-digit token that was not issued for the submitted reset workflow.	400 Bad Request	PASS — API-014 - reviewed status is 400	N/A
API-015	`PASS`	Verify that a valid OTP issued for one email cannot be paired with a different email.	400 Bad Request	PASS — API-015 - reviewed status is 400	N/A
API-016	`FAIL_ASSERTION`	Verify that omission of reviewed-required member newPassword is rejected.	200 OK	FAIL — API-016 - reviewed status is 400	expected response to have status code 400 but got 200
API-017	`FAIL_ASSERTION`	Verify that null is rejected for reviewed-required string member newPassword.	200 OK	FAIL — API-017 - reviewed status is 400	expected response to have status code 400 but got 200

API-018	`FAIL_ASSERTION`	Verify that a non-string JSON value is rejected for newPassword without coercion.	200 OK	FAIL — API-018 - reviewed status is 400	expected response to have status code 400 but got 200
API-019	`PASS`	Verify acceptance of a password at the documented minimum length that contains every required character class.	200 OK	PASS — API-019 - reviewed status is 200	N/A
API-020	`FAIL_ASSERTION`	Verify rejection of a seven-character password even when all required character classes are present.	200 OK	FAIL — API-020 - reviewed status is 400	expected response to have status code 400 but got 200
API-021	`FAIL_ASSERTION`	Verify rejection of an otherwise conforming password that lacks an uppercase letter.	200 OK	FAIL — API-021 - reviewed status is 400	expected response to have status code 400 but got 200
API-022	`FAIL_ASSERTION`	Verify rejection of an otherwise conforming password that lacks a lowercase letter.	200 OK	FAIL — API-022 - reviewed status is 400	expected response to have status code 400 but got 200
API-023	`FAIL_ASSERTION`	Verify rejection of an otherwise conforming password that lacks a digit.	200 OK	FAIL — API-023 - reviewed status is 400	expected response to have status code 400 but got 200
API-024	`FAIL_ASSERTION`	Verify rejection of a password whose only punctuation is outside the allowed special-character set.	200 OK	FAIL — API-024 - reviewed status is 400	expected response to have status code 400 but got 200

API-025	`BLOCKED_NOT_EXECUTED`	Verify that an expired OTP cannot satisfy the reset request contract.	N/A	N/A — not executed	N/A
API-026	`PASS`	Setup: pre-request first-use reset (`pm.sendRequest` , expected HTTP 200) Verify that an OTP already used in a successful reset cannot satisfy a second reset request.	Setup: unavailable in detailed execution rows 400 Bad Request	Setup: no Newman assertion PASS — API-026 - reviewed status is 400	N/A
API-078	`FAIL_ASSERTION`	Step 1: Verify that otherwise valid JSON text sent as text/plain receives a 4xx client-error response and is rejected without consuming the OTP. [Step 1 - text/plain] Step 2: Verify that otherwise valid JSON text sent as text/plain receives a 4xx client-error response and is rejected without consuming the OTP. [Step 2 - JSON retry]	Step 1: 500 Internal Server Error Step 2: 200 OK	Step 1: FAIL — API-078 - text/plain response is 4xx Step 2: PASS — API-078 - reviewed status is 200	Step 1: expected 500 to be within 400..499
API-079	`PASS`	Step 1: Verify that an empty application/json request body is rejected safely without consuming the OTP. [Step 1 - empty body] Step 2: Verify that an empty application/json request body is rejected safely without consuming the OTP. [Step 2 - valid retry]	Step 1: 400 Bad Request Step 2: 200 OK	Step 1: PASS — API-079 - reviewed status is 400 Step 2: PASS — API-079 - reviewed status is 200	N/A

API-027	PASS	Verify the documented JSON object shape with a registered email, its usable six-digit OTP, and a conforming password.	200 OK	PASS — API-027 - reviewed status is 200	N/A
API-028	PASS	Verify rejection of a well-formed JSON request whose top-level value is an array rather than an object.	400 Bad Request	PASS — API-028 - reviewed status is 400	N/A
API-029	PASS	Verify rejection of a malformed JSON request body.	400 Bad Request	PASS — API-029 - reviewed status is 400	N/A
API-030	PASS	EXPLORATORY/CHARACTERIZATION: Record how the endpoint handles an additional JSON member while all documented members remain valid.	200 OK	NO_AUTOMATED_ASSERTIONS	N/A
API-031	PASS	Verify that an OTP cannot be used with a different registered email.	400 Bad Request	PASS — API-031 - reviewed status is 400	N/A
API-032	PASS	Verify handling of a syntactically ordinary email address that is not registered.	400 Bad Request	PASS — API-032 - reviewed status is 400	N/A
API-033	PASS	Verify rejection of a clearly malformed email string without imposing a restrictive email regex.	400 Bad Request	PASS — API-033 - reviewed status is 400	N/A
API-034	PASS	Verify rejection of an empty email string.	400 Bad Request	PASS — API-034 - reviewed status is 400	N/A

API-035	`PASS`	Verify rejection when the required email member is omitted.	400 Bad Request	PASS — API-035 - reviewed status is 400	N/A
API-036	`PASS`	Verify rejection when email is JSON null.	400 Bad Request	PASS — API-036 - reviewed status is 400	N/A
API-037	`PASS`	Verify rejection when email has a non-string JSON type.	400 Bad Request	PASS — API-037 - reviewed status is 400	N/A
API-038	`PASS`	Verify rejection of an empty resetToken string.	400 Bad Request	PASS — API-038 - reviewed status is 400	N/A
API-039	`PASS`	Verify the just-below OTP length boundary with five decimal digits.	400 Bad Request	PASS — API-039 - reviewed status is 400	N/A
API-040	`PASS`	Verify the just-above exact OTP length boundary with seven decimal digits.	400 Bad Request	PASS — API-040 - reviewed status is 400	N/A
API-041	`PASS`	Verify rejection of a six-character resetToken containing a non-decimal character.	400 Bad Request	PASS — API-041 - reviewed status is 400	N/A
API-042	`PASS`	Verify rejection when the required resetToken member is omitted.	400 Bad Request	PASS — API-042 - reviewed status is 400	N/A
API-043	`PASS`	Verify rejection when resetToken is JSON null.	400 Bad Request	PASS — API-043 - reviewed status is 400	N/A
API-044	`FAIL_ASSERTION`	Verify rejection when a valid issued resetToken value is represented as a JSON number rather than a string.	200 OK	FAIL — API-044 - reviewed status is 400	expected response to have status code 400 but got 200

API-045	`PASS`	Verify rejection of a well-formed six-digit token that was not issued for the baseline email.	400 Bad Request	PASS — API-045 - reviewed status is 400	N/A
API-046	`FAIL_ASSERTION`	Verify rejection of an empty newPassword string.	200 OK	FAIL — API-046 - reviewed status is 400	expected response to have status code 400 but got 200
API-047	`FAIL_ASSERTION`	Verify the just-below password length boundary with a seven-character password that otherwise contains all required classes.	200 OK	FAIL — API-047 - reviewed status is 400	expected response to have status code 400 but got 200
API-048	`PASS`	Verify the eight-character password boundary with exactly one uppercase, one digit, and one allowed special character.	200 OK	PASS — API-048 - reviewed status is 200	N/A
API-049	`PASS`	Verify the just-above password length boundary with nine characters while all composition rules remain satisfied.	200 OK	PASS — API-049 - reviewed status is 200	N/A
API-050	`FAIL_ASSERTION`	Verify the just-below uppercase-count boundary with zero uppercase letters while all other password clauses remain satisfied.	200 OK	FAIL — API-050 - reviewed status is 400	expected response to have status code 400 but got 200
API-051	`PASS`	Verify the just-above uppercase-count boundary with two uppercase letters while all password clauses remain satisfied.	200 OK	PASS — API-051 - reviewed status is 200	N/A
API-052	`FAIL_ASSERTION`	Verify the just-below lowercase-count boundary with zero lowercase letters while all other password clauses remain satisfied.	200 OK	FAIL — API-052 - reviewed status is 400	expected response to have status code 400 but got 200
API-053	`PASS`	Verify the lowercase-count boundary with exactly one lowercase letter while all password clauses remain satisfied.	200 OK	PASS — API-053 - reviewed status is 200	N/A

API-054	`PASS`	Verify the just-above lowercase-count boundary with exactly two lowercase letters while all password clauses remain satisfied.	200 OK	PASS — API-054 - reviewed status is 200	N/A
API-055	`FAIL_ASSERTION`	Verify the just-below digit-count boundary with zero digits while all other password clauses remain satisfied.	200 OK	FAIL — API-055 - reviewed status is 400	expected response to have status code 400 but got 200
API-056	`PASS`	Verify the just-above digit-count boundary with exactly two digits while all password clauses remain satisfied.	200 OK	PASS — API-056 - reviewed status is 200	N/A
API-057	`FAIL_ASSERTION`	Verify the just-below allowed-special-count boundary with zero listed special characters while all other password clauses remain satisfied.	200 OK	FAIL — API-057 - reviewed status is 400	expected response to have status code 400 but got 200
API-058	`PASS`	Verify the just-above allowed-special-count boundary with exactly two listed special characters while all password clauses remain satisfied.	200 OK	PASS — API-058 - reviewed status is 200	N/A
API-059	`PASS`	Verify successful reset with a valid password that satisfies every explicit strength clause and also contains an additional non-listed punctuation character.	200 OK	PASS — API-059 - reviewed status is 200	N/A
API-060	`FAIL_ASSERTION`	Verify rejection when the required newPassword member is omitted.	200 OK	FAIL — API-060 - reviewed status is 400	expected response to have status code 400 but got 200
API-061	`FAIL_ASSERTION`	Verify rejection when newPassword is JSON null.	200 OK	FAIL — API-061 - reviewed status is 400	expected response to have status code 400 but got 200

API-062	`FAIL_ASSERTION`	Verify rejection when newPassword has a non-string JSON type.	200 OK	FAIL — API-062 - reviewed status is 400	expected response to have status code 400 but got 200
API-080	`FAIL_ASSERTION`	Step 1: Verify that an object-valued newPassword is rejected without consuming the OTP. [Step 1 - object password] Step 2: Verify that an object-valued newPassword is rejected without consuming the OTP. [Step 2 - string retry]	Step 1: 200 OK Step 2: 400 Bad Request	Step 1: FAIL — API-080 - reviewed status is 400 Step 2: FAIL — API-080 - reviewed status is 200	Step 1: expected response to have status code 400 but got 200 Step 2: expected response to have status code 200 but got 400
API-063	`PASS`	Verify reviewed transition TR-001: a usable OTP for its bound email completes the intended reset and becomes invalidated after use (ST-01 to ST-03).	200 OK	PASS — API-063 - reviewed status is 200	N/A
API-064	`PASS`	Verify reviewed invalid transition TR-002: an ST-01 OTP bound to email A cannot be used with a different email B.	400 Bad Request	PASS — API-064 - reviewed status is 400	N/A
API-065	`BLOCKED_NOT_EXECUTED`	Verify reviewed invalid transition TR-003: an expired OTP in ST-02 cannot perform a valid password reset.	N/A	N/A — not executed	N/A

API-066	`PASS`	Setup: pre-request first-use reset (`pm.sendRequest` , expected HTTP 200) Verify reviewed invalid transition TR-004: replaying an OTP after it reached ST-03 cannot perform another reset.	Setup: unavailable in detailed execution rows 400 Bad Request	Setup: no Newman assertion PASS — API-066 - reviewed status is 400	N/A
API-067	`FAIL_ASSERTION`	Verify reviewed guard-failing transition TR-005: rejecting a weak new password preserves the usable OTP in ST-01 and leaves the account's old password unchanged.	200 OK	FAIL — API-067 - reviewed status is 400	expected response to have status code 400 but got 200
API-077	`PASS`	Two synchronized reset requests using the same email/OTP but different valid passwords.	One request: 200 OK Other request: 400 Bad Request	MANUAL PASS	N/A

API-068	`FAIL`	WHITE-BOX/STORAGE VERIFICATION: Verify that a successfully reset password is not persisted as plaintext.	200 OK	MANUAL FAIL	The submitted reset password was stored in plaintext.
API-069	`PASS`	Verify that inert SQL/query metacharacters in a client-controlled field do not bypass reset controls, affect another account, expose database information, or cause unexpected server failure.	400 Bad Request	PASS — API-069 - reviewed status is 400	N/A
API-070	`PASS`	Verify the valid security control: a random exactly 6-decimal-digit, unexpired, unused OTP authorizes reset only for the email to which it is bound and is consumed on successful use.	200 OK	PASS — API-070 - reviewed status is 200	N/A
API-071	`PASS`	Verify that an OTP issued for one email cannot authorize a reset for another email.	400 Bad Request	PASS — API-071 - reviewed status is 400	N/A
API-072	`BLOCKED_NOT_EXECUTED`	Verify that an expired reset OTP cannot authorize a password reset.	N/A	N/A — not executed	N/A

API-073	`PASS`	Setup: pre-request first-use reset (`pm.sendRequest` , expected HTTP 200) Verify one-time use by rejecting replay of an OTP after it has successfully reset the bound account's password.	Setup: unavailable in detailed execution rows 400 Bad Request	Setup: no Newman assertion PASS — API-073 - reviewed status is 400	N/A
API-074	`PASS`	Verify that a resetToken shorter than the specified six decimal digits cannot authorize reset.	400 Bad Request	PASS — API-074 - reviewed status is 400	N/A
API-075	`BLOCKED_NOT_EXECUTED`	Verify resistance to automated guessing of six-digit reset OTPs through rate limiting or equivalent abuse protection.	N/A	N/A — not executed	N/A
API-076	`PASS`	Step 1: Verify that reset failure behavior does not materially disclose whether the submitted account exists. [Request A - registered email] Step 2: Verify that reset failure behavior does not materially disclose whether the submitted account exists. [Request B - non-existing email]	Step 1: 400 Bad Request Step 2: 400 Bad Request	Step 1: PASS — API-076 - reviewed status is 400 Step 2: PASS — API-076 - reviewed status is 400 Step 2: PASS — API-076 - comparable failure representation	N/A

API-081	`PASS`	Step 1: Verify that SQL syntax in a rule-conforming new password is treated as data and does not damage another account's reset path. [Step 1 - Account A injection string] Step 2: Verify that SQL syntax in a rule-conforming new password is treated as data and does not damage another account's reset path. [Step 2 - Account B integrity check]	Step 1: 200 OK Step 2: 200 OK	Step 1: PASS — API-081 - reviewed status is 200 Step 2: PASS — API-081 - reviewed status is 200	N/A
API-082	`PASS`	Verify that an invalid bearer token does not interfere with the public password-recovery endpoint.	200 OK	PASS — API-082 - reviewed status is 200	N/A

Coverage and reconciliation notes

- The embedded collection supplied the authoritative intended set: 88 leaf request items grouped into 82 stable Test IDs. All observed main requests matched collection items by immutable item ID; no ambiguous or unmatched execution remained.
- Four logical cases remain explicitly blocked before dispatch: `API-025`, `API-065`, and `API-072` because no observable/configurable OTP-expiry state is available, and `API-075` because no authoritative abuse-control threshold is available.
- `API-026`, `API-066`, and `API-073` each issue a first-use reset through pre-request `pm.sendRequest` before the collection replay request. These three helper calls reconcile Newman’s 85 request total with 82 executed collection leaf requests. The JSON repeats each main execution object twice with identical cursor/request/response/assertion identity; each duplicate pair was correlated once for logical-case and assertion analysis. Newman’s source assertion totals then reconcile exactly to 82 assertions: 59 passed and 23 failed.
- `API-030` completed with HTTP 200 but defined no automated assertion; its `PASS` status means execution completed without request/runtime error. Final review classifies it as `exploratory` because the specification provides no additional-property oracle.
- `API-080` contains two failed assertions across its two ordered steps, so the run has 23 failed assertions but 22 distinct logical cases with failed assertions.
- Manual-oracle requirements are independent of automated status. Automated results do not satisfy the explicitly retained checks, including storage inspection, password/OTP state verification, side-effect review, configured abuse-control execution, and true concurrency.
- `API-077` was blocked only in sequential Newman execution, then separately verified with a synchronized concurrency harness and received a final `PASS`.
- `API-068` passed its automated HTTP assertion but failed the required manual storage oracle; its final reviewed result is `FAIL` because the reset password was confirmed to be stored in plaintext.

Pool B: Discount Coupons

B.1. Introduction

Pool B covers **FR-09 – Discount Coupons**. This report selects `POST /api/apply-coupon`, which applies a coupon to an order amount and returns the calculated `discount_amount` and `final_amount`.

B.2. Contractual Testing Analysis

The selected endpoint is documented as `POST /api/apply-coupon`. It uses a JSON body with three top-level fields and requires a valid JWT under FR-09 C4:

Input	Documented representation	Reviewed constraint
`Authorization`	`Bearer <token>` header	The JWT must be valid; token claims and authentication-failure responses are not specified
`code`	JSON string	The coupon must exist and have `is_active = 1` to satisfy FR-09 C1
`total_amount`	JSON number	Must be greater than or equal to the selected coupon's `min_order_amount`
`user_id`	JSON number	The user's prior use count for the coupon must be below `max_uses_per_user`

The reviewed contract model contains rules `CR-001` – `CR-012`. It covers the method and path, documented JSON representation, the three body members, valid-JWT prerequisite, successful response structure, percent and fixed discount calculations, final-amount calculation, and exact calculation oracles for the documented `SAVE10` and `BIGBUY` fixtures.

On a qualifying request, the response is documented as JSON containing `discount_amount` and `final_amount`. For a percent coupon, `discount_amount = total × discount_value / 100`; for a fixed coupon, `discount_amount = discount_value`; and `final_amount = total - discount_amount`.

The API specification does not provide a formal request or response schema. Field requiredness, nullability, coercion, additional-property handling, concrete MIME values, HTTP statuses, error bodies, response headers, rounding, and validation precedence are not documented. The reviewed model therefore treats malformed bodies, omitted members, wrong types, and similar mutations as bounded contract explorations without inventing rejection statuses or messages. It also does not infer that body `user_id` must match the JWT subject. The complete analysis is stored in [`review/pool-b/reports/contract-report.md`](review/pool-b/reports/contract-report.md).

B.3. Domain Testing Analysis

Domain analysis uses a qualifying `SAVE10` request as its baseline: the coupon is arranged as active and unexpired, `total_amount` is `500000`, the selected user has zero prior uses, and a valid JWT is supplied. This produces the mathematical oracle `discount_amount = 50000` and `final_amount = 450000`. Tests vary one input or one controlled coupon/user condition at a time where possible.

The reviewed model defines 37 equivalence partitions (`DP-001` – `DP-037`) across the JSON container, JWT header, and three body fields. Important partitions include:

- JSON object, malformed/non-object body, and additional-member representations;
- valid, missing, malformed, invalid, and expired JWT values;
- existing active percent/fixed coupons, nonexistent or inactive coupons, and coupons before, at, or after expiry;
- totals below, equal to, and above the selected coupon's minimum, including zero, negative, decimal, very large, wrong-type, omitted, and `null` values;
- per-user use counts below, at, and above `max_uses_per_user`;
- matching or mismatched JWT/body identities, nonexistent users, unusual identifier values, wrong types, omission, and `null`.

Seven boundary models (`DB-001` – `DB-007`) cover:

Boundary group	Reviewed points
`SAVE10` / `VIP100` minimum `300000`	`299999` outside; `300000` and `300001` inside
`BIGBUY` minimum `500000`	`499999` outside; `500000` and `500001` inside
Minimum allowed coupon threshold `0`	`-1` outside; `0` and `1` inside
Expiry date	before expiry qualifies; equality and after expiry do not qualify
Generic and fixture-specific usage limits	`M - 1` qualifies; `M` and `M + 1` do not qualify

Coupon activity, expiry, and prior-use count are state-dependent eligibility preconditions, but they remain DOMAIN coverage because this endpoint documents calculation rather than a state change. The analysis does not infer integer-only amounts, currency precision, rounding, maximum discounts, a zero floor for `final_amount`, identifier ranges, case normalization, or undocumented error responses. The complete analysis is stored in `review/pool-b/reports/domain-report.md`.

B.4. State Transition Testing Analysis

State-transition testing is **not applicable** to the selected endpoint. FR-09 C1, C2, and C5 make coupon eligibility depend on stored conditions—active/inactive status, expiry, and the user's prior-use count—but the API specification describes `POST /api/apply-coupon` only as calculating discounted amounts.

Neither authoritative source states that this call increments coupon usage, consumes or reserves a coupon, changes `is_active`, creates a destination state, follows a lifecycle sequence, or has an idempotency rule. A transition model would therefore invent behavior. Active/inactive, before/equal/after-expiry, and below/at/above-use-limit cases are covered under DOMAIN instead. The Phase-2 STATE candidate fragment is exactly `[]`. The applicability record is stored in `review/pool-b/reports/state-report.md`.

B.5. Security Testing Analysis

All supplied requirements `SEC-01` – `SEC-07` were evaluated for this endpoint:

Requirement	Applicability and treatment
`SEC-01`	Not applicable: the endpoint has no password input, output, or storage behavior.
`SEC-02`	Applicable: FR-09 C4 explicitly requires a valid JWT. Coverage includes a valid control and missing, malformed, invalid-signature, and expired JWT classes.
`SEC-03`	Not applicable: this Checkout operation is not an Admin API.
`SEC-04`	No direct API scenario: no UI rendering or response-reflection sink is documented for coupon input.
`SEC-05`	Applicable at the query/implementation layer: controlled inert SQL/metacharacter values are applied to both `code` and client-controlled `user_id`.
`SEC-06`	Not applicable: this is not a profile-update API and has no documented `role` input.
`SEC-07`	Not applicable: the endpoint has no password-reset OTP behavior.

The reviewed security model contains eight scenarios (`SS-001` – `SS-008`). `SS-001` – `SS-005` cover the valid JWT control and missing or invalid credential classes. `SS-006` – `SS-008` cover inert query-like values in coupon `code` and `user_id`; these values must remain data and must not alter query semantics, broaden the per-user usage scope, select another user's record, modify database state, or expose query diagnostics.

No HTTP status, error schema, or message is invented for the negative security scenarios. Black-box results may reveal unsafe behavior but cannot conclusively prove that parameterized queries are used; source review, query instrumentation, or equivalent implementation evidence is required for that claim. The sources also do not define JWT claims, issuer, audience, accepted algorithms, clock skew, or a binding between the JWT identity and body `user_id`. The complete analysis is stored in `review/pool-b/reports/security-report.md`.

B.6. AI-Generated Test Cases

The final reviewed test cases for this pool are stored in: `test-cases/b-discount-coupons.csv`. The audited AI-generated subset is stored in `review/pool-b/candidate-api-tests.csv`.

The AI-generated test cases were derived from the reviewed analyses in Sections B.2–B.5, then seven human-authored cases from Section B.7 were merged into the final CSV.

Testing Type	AI-generated	Human-added	Final total
Contractual Testing	15	2	17

Domain Testing	44	4	48
State Transition Testing	0	0	0
Security Testing	8	1	9
Total	**67**	**7**	**74**

The final suite uses sequential IDs `API-001` – `API-074` and exactly nine traceability fields per record. Every case targets `POST /api/apply-coupon`. The 67-case AI subset covers `CR-001` – `CR-012`, `DP-001` – `DP-037`, `DB-001` – `DB-007`, and `SS-001` – `SS-008`; applicable security requirements are `SEC-02` and `SEC-05`. No placeholder STATE cases were generated. The review artifact preserves the same first nine fields for all AI-generated cases and adds an explicit `VALID` audit result with a brief case-level reason; no invalid or incomplete case remains in the retained AI subset.

Human review corrected inclusive minimum-threshold labels, the mathematical oracle for `SAVE10` with `total_amount = 300001`, and the fixture preconditions for `BIGBUY`, `EXPIRED`, and nonexistent coupon codes. Same-category semantic deduplication removed 14 redundant DOMAIN records while preserving every partition/boundary basis and provisional specialist ID on retained cases. The final merged coverage includes `DP-021 + DB-001` just-below, `DP-013 + DB-004` equal-to-expiry, and `DP-030/DP-031 + DB-006` at/above the usage limit.

Final validation confirmed 74 sequential unique IDs, the exact nine-column final schema, endpoint/category consistency, non-empty required content, complete reviewed AI traceability, and no duplicate IDs or rows. The final Postman collection was generated and executed with Newman; execution results and defect triage are reported in Section B.8.

B.7. Human Cases

ID	Category	Test case	Expected result	Notes
API-068	CONTRACT	Send otherwise valid JSON text using `Content-Type: text/plain`.	The request does not produce a successful coupon calculation. It is handled safely without a `5xx` server error.	The specification documents a JSON request body but does not define behavior for unsupported content types. The safe rejection expectation is human-added.
API-069	CONTRACT	Send `Content-Type: application/json` with a completely **empty HTTP body**.	The request does not apply a coupon and does not return a successful calculation. It is handled safely without a `5xx` server error.	Previous AI cases tested missing fields and malformed JSON, but not a completely missing body.
API-070	DOMAIN	Send an empty coupon code `""` while all other request conditions are valid.	The coupon is not applied because the submitted code does not identify an existing active coupon.	The AI tested unusual, null, numeric, and modified coupon-code values, but did not use the empty string as a separate representative.
API-071	DOMAIN	Apply `VIP100` with `total_amount = 300000`, exactly equal to its documented minimum, with zero prior uses.	The coupon qualifies. `discount_amount = 100000` and `final_amount = 200000`.	The generated boundary cases covered SAVE10 and BIGBUY minimum amounts, but did not test the exact minimum of VIP100.
API-072	SECURITY	With a valid JWT for user 1 and body `user_id = 0`, observe whether usage eligibility is scoped to the JWT identity or the body value.	Record whether the coupon is applied or rejected; neither outcome is an FR-09 identity-binding failure because no JWT/body binding rule is specified.	The original rejection expectation was withdrawn as an unsupported human assumption. The case is now exploratory and has no automated outcome oracle.

API-073	DOMAIN	Arrange an active percent coupon with `discount_value = 1`, `min_order_amount = 0`, and remaining usage. Apply it to `total_amount = 500000`.	`discount_amount = 5000` and `final_amount = 495000`.	The AI tested the documented percentage coupon but did not test the lower positive percentage boundary allowed by the documented coupon rules.
API-074	DOMAIN	Arrange an active percent coupon with `discount_value = 100`, `min_order_amount = 0`, and remaining usage. Apply it to `total_amount = 500000`.	`discount_amount = 500000` and `final_amount = 0`.	No upper percentage limit is documented. This case checks the percentage formula at a full-discount value.

The AI missed these cases mainly because the initial generation focused on representative partitions and boundaries that could be derived directly from the API specification. Some additional request representations, such as an unsupported content type and a completely empty body, were therefore not selected.

The generated domain tests also covered the main documented coupon fixtures and representative boundaries, but did not systematically test every documented coupon at its own threshold or additional valid percentage values such as 1% and 100%.

The security analysis tested JWT validity, SQL-like inputs, and per-user usage limits, but did not combine authentication identity with a manipulated `user_id` to characterize JWT/body-user interaction. Human review added that combination, then withdrew the original rejection assumption because the specification does not define an identity-binding rule; `API-072` is therefore exploratory.

These gaps show that specification-based AI generation can provide broad coverage, but human review is still useful for finding robustness cases, additional boundary combinations, and security interactions between otherwise separate input conditions.

The seven human-authored cases above are merged into the final CSV as `API-068` – `API-074`.

B.8. Newman Execution Analysis

Scope and evidence

This analysis uses the real Newman JSON artifact `reports/pool-b/pool-b-run.json`, the 74 reviewed cases in `test-cases/b-discount-coupons.csv`, the generated collection in `postman/pool-b-discount-coupons.postman_collection.json`, and `reference/api_specification.md`. Pool A Section A.8 supplies the presentation format. The original full-run evidence remains unchanged; the later triage resolution corrected only the faulty `API-067` assertion and the unsupported `API-072` human oracle.

Execution status and defect triage are deliberately separate. `FAIL_ASSERTION` means that a reviewed automated oracle failed; it does not by itself prove an SUT defect. Likewise, `PASS` with `NO_AUTOMATED_ASSERTIONS` confirms only that the request completed. The later final-review resolution supplies the conformance, blocked, or exploratory disposition.

Run summary

Metric	Normalized result
Collection	`Pool B - Discount Coupon Reviewed Tests`
Execution window	2026-08-22 18:04:32.417 to 18:04:38.117 (GMT+7)
Duration	5.700 seconds
Reviewed collection requests / logical cases	74 / 74; every reviewed ID executed
Total requests	148 in Newman `run.stats`: 74 reviewed SUT requests plus 74 fixture-reset helper calls
Newman tests / scripts	74 tests; 74 test scripts; 74 pre-request scripts; 148 total script executions

Assertions	27 total: 7 passed, 20 failed, 0 pending/skipped
Request, pre-request-script, and test-script errors	0 / 0 / 0
Logical execution statuses	`PASS`: 54; `FAIL_ASSERTION`: 20; `REQUEST_ERROR`: 0; `RUNTIME_ERROR`: 0; `NOT_EXECUTED`: 0
PASS detail	7 cases passed assertions; 47 cases completed with `NO_AUTOMATED_ASSERTIONS`
Explicit manual-oracle requirements after final review	50 cases: 48 manual/exploratory-only cases (including corrected `API-072`) plus `API-065` and `API-066`, whose black-box assertions do not prove all required database/query effects; `API-067` is complete through implementation and database-state evidence
Failed-case triage after final review	`SUT_BUG`: 18; `TEST_DEFECT`: 2 (`API-067`, resolved; `API-072`, unsupported assumption); `NEEDS_MANUAL_REVIEW`: 0; `SETUP_DEFECT`: 0
Additional suspicious manual-only executions	6 `SUT_BUG` candidates (`API-013`, `API-014`, `API-020`, `API-021`, `API-035`, `API-047`)
Total SUT bug-candidate case observations	24 (18 failed assertions plus 6 suspicious manual-only executions; this is a case count, not a count of unique root causes)

Logical test-case outcomes

Every row below is the single `POST /api/apply-coupon` step unless the reviewed case deliberately changes the method or representation. All items matched by immutable collection item/request identity and executed once. The JSON reporter contains two identical detailed representations per SUT request with the same `httpRequestId`; these duplicates were correlated and were not counted twice.

Test ID	Execution Status	HTTP Status	Assertion Result	Manual Oracle Required
API-001	`FAIL_ASSERTION`	200 OK	FAIL — expected `discount_amount=50000`; got `-4500000`	NO
API-002	`FAIL_ASSERTION`	400 Bad Request	FAIL — expected fixed-coupon calculation; response had no calculation fields	NO
API-003	`FAIL_ASSERTION`	400 Bad Request	FAIL — expected `discount_amount=30000`; response had no calculation fields	NO
API-004	`PASS`	404 Not Found	`NO_AUTOMATED_ASSERTIONS`	YES — exploratory method handling
API-005	`PASS`	400 Bad Request	`NO_AUTOMATED_ASSERTIONS`	YES — malformed JSON handling
API-006	`PASS`	400 Bad Request	`NO_AUTOMATED_ASSERTIONS`	YES — non-object JSON handling
API-007	`PASS`	400 Bad Request	`NO_AUTOMATED_ASSERTIONS`	YES — omitted `code` handling
API-008	`PASS`	404 Not Found	`NO_AUTOMATED_ASSERTIONS`	YES — non-string `code` handling
API-009	`PASS`	400 Bad Request	`NO_AUTOMATED_ASSERTIONS`	YES — omitted `total_amount` handling

API-010	`PASS`	200 OK	`NO_AUTOMATED_ASSERTIONS`	YES — numeric-string coercion
API-011	`PASS`	200 OK	`NO_AUTOMATED_ASSERTIONS`	YES — omitted `user_id` behavior
API-012	`PASS`	200 OK	`NO_AUTOMATED_ASSERTIONS`	YES — string `user_id` coercion
API-013	`PASS`	200 OK	`NO_AUTOMATED_ASSERTIONS`	YES — missing-JWT rejection semantics
API-014	`PASS`	200 OK	`NO_AUTOMATED_ASSERTIONS`	YES — invalid-JWT rejection semantics
API-015	`PASS`	200 OK	`NO_AUTOMATED_ASSERTIONS`	YES — additional-property handling
API-068	`FAIL_ASSERTION`	500 Internal Server Error	FAIL — expected status below 500	NO
API-069	`PASS`	400 Bad Request	PASS — status below 500	NO
API-016	`FAIL_ASSERTION`	200 OK	FAIL — expected `discount_amount=50000`; got `-4500000`	NO
API-017	`PASS`	400 Bad Request	`NO_AUTOMATED_ASSERTIONS`	YES — array-body handling
API-018	`PASS`	400 Bad Request	`NO_AUTOMATED_ASSERTIONS`	YES — malformed JSON handling
API-019	`PASS`	200 OK	`NO_AUTOMATED_ASSERTIONS`	YES — extra-member handling
API-020	`PASS`	200 OK	`NO_AUTOMATED_ASSERTIONS`	YES — missing-JWT behavior
API-021	`PASS`	200 OK	`NO_AUTOMATED_ASSERTIONS`	YES — invalid-JWT behavior
API-022	`FAIL_ASSERTION`	400 Bad Request	FAIL — expected fixed-coupon calculation; response had no calculation fields	NO
API-023	`PASS`	404 Not Found	`NO_AUTOMATED_ASSERTIONS`	YES — nonexistent coupon response
API-024	`PASS`	404 Not Found	`NO_AUTOMATED_ASSERTIONS`	YES — inactive coupon response
API-025	`PASS`	400 Bad Request	`NO_AUTOMATED_ASSERTIONS`	YES — expiry-date equality behavior
API-026	`PASS`	400 Bad Request	`NO_AUTOMATED_ASSERTIONS`	YES — expired coupon response
API-027	`PASS`	404 Not Found	`NO_AUTOMATED_ASSERTIONS`	YES — unusual code handling

API-028	`PASS`	404 Not Found	`NO_AUTOMATED_ASSERTIONS`	YES — whitespace/case handling
API-029	`PASS`	400 Bad Request	`NO_AUTOMATED_ASSERTIONS`	YES — null code handling
API-030	`PASS`	404 Not Found	`NO_AUTOMATED_ASSERTIONS`	YES — numeric code handling
API-031	`FAIL_ASSERTION`	400 Bad Request	FAIL — expected `discount_amount=30000`; response had no calculation fields	NO
API-032	`PASS`	400 Bad Request	`NO_AUTOMATED_ASSERTIONS`	YES — below-minimum response
API-033	`FAIL_ASSERTION`	400 Bad Request	FAIL — expected zero calculation at minimum 0; response had no calculation fields	NO
API-034	`PASS`	400 Bad Request	`NO_AUTOMATED_ASSERTIONS`	YES — negative-total handling
API-035	`PASS`	200 OK	`NO_AUTOMATED_ASSERTIONS`	YES — numeric precision/formula observation
API-036	`PASS`	200 OK	`NO_AUTOMATED_ASSERTIONS`	YES — numeric-string coercion
API-037	`PASS`	400 Bad Request	`NO_AUTOMATED_ASSERTIONS`	YES — null-total handling
API-038	`PASS`	400 Bad Request	`NO_AUTOMATED_ASSERTIONS`	YES — boolean-total handling
API-039	`PASS`	400 Bad Request	`NO_AUTOMATED_ASSERTIONS`	YES — non-JSON numeric token handling
API-040	`PASS`	400 Bad Request	`NO_AUTOMATED_ASSERTIONS`	YES — usage-limit boundary response
API-041	`PASS`	400 Bad Request	`NO_AUTOMATED_ASSERTIONS`	YES — above-limit response
API-042	`PASS`	200 OK	`NO_AUTOMATED_ASSERTIONS`	YES — JWT/body-user mismatch semantics
API-043	`PASS`	200 OK	`NO_AUTOMATED_ASSERTIONS`	YES — nonexistent-user behavior
API-044	`PASS`	200 OK	`NO_AUTOMATED_ASSERTIONS`	YES — fractional `user_id` handling
API-045	`PASS`	200 OK	`NO_AUTOMATED_ASSERTIONS`	YES — null `user_id` handling
API-046	`PASS`	200 OK	`NO_AUTOMATED_ASSERTIONS`	YES — string `user_id` handling
API-047	`PASS`	200 OK	`NO_AUTOMATED_ASSERTIONS`	YES — numeric precision/formula observation
API-048	`PASS`	400 Bad Request	`NO_AUTOMATED_ASSERTIONS`	YES — below-minimum response

API-049	`PASS`	200 OK	PASS — `discount_amount=50000`, `final_amount=450001`	NO
API-050	`PASS`	400 Bad Request	`NO_AUTOMATED_ASSERTIONS`	YES — below-zero-minimum response
API-051	`FAIL_ASSERTION`	200 OK	FAIL — expected `discount_amount=0.1`; got `-9`	NO
API-052	`FAIL_ASSERTION`	200 OK	FAIL — expected `discount_amount=50000`; got `-4500000`	NO
API-053	`PASS`	400 Bad Request	`NO_AUTOMATED_ASSERTIONS`	YES — after-expiry response
API-054	`PASS`	200 OK	PASS — `discount_amount=50000`, `final_amount=450000`	NO
API-055	`PASS`	400 Bad Request	`NO_AUTOMATED_ASSERTIONS`	YES — at-limit response
API-056	`PASS`	400 Bad Request	`NO_AUTOMATED_ASSERTIONS`	YES — above-limit response
API-057	`PASS`	200 OK	PASS — `discount_amount=100000`, `final_amount=400000`	NO
API-058	`PASS`	400 Bad Request	`NO_AUTOMATED_ASSERTIONS`	YES — at-limit response
API-059	`PASS`	400 Bad Request	`NO_AUTOMATED_ASSERTIONS`	YES — above-limit response
API-070	`PASS`	400 Bad Request	PASS — no successful calculation returned	NO
API-071	`FAIL_ASSERTION`	400 Bad Request	FAIL — expected `discount_amount=100000`; response had no calculation fields	NO
API-073	`FAIL_ASSERTION`	200 OK	FAIL — expected `discount_amount=5000`; got `0`	NO
API-074	`FAIL_ASSERTION`	200 OK	FAIL — expected `discount_amount=500000`; got `-49500000`	NO
API-060	`FAIL_ASSERTION`	200 OK	FAIL — expected `discount_amount=50000`; got `-4500000`	NO
API-061	`FAIL_ASSERTION`	200 OK	FAIL — successful calculation returned without JWT	NO
API-062	`FAIL_ASSERTION`	200 OK	FAIL — successful calculation returned for malformed JWT	NO
API-063	`FAIL_ASSERTION`	200 OK	FAIL — successful calculation returned for invalid-signature JWT	NO
API-064	`FAIL_ASSERTION`	200 OK	FAIL — successful calculation returned for expired JWT	NO

API-065	`PASS`	404 Not Found	PASS — no successful calculation returned	YES — query structure/diagnostics/state require external inspection
API-066	`PASS`	404 Not Found	PASS — no successful calculation returned	YES — parameterization and persistent state require external inspection
API-067	`PASS` — targeted rerun	200 OK	PASS — response exposes no database query diagnostics	NO — bound-query implementation and unchanged before/after database snapshot complete the non-black-box checks
API-072	`FAIL_ASSERTION` — historical full-run result; oracle withdrawn	200 OK	Historical FAIL — the old assertion required rejection. Current exploratory case has `NO_AUTOMATED_ASSERTIONS` and was not rerun.	YES — record the unspecified JWT/body identity behavior without a conformance verdict

Failed and suspicious-case triage

The following table classifies every failed assertion individually. Causes describe the evidence-level mismatch, not an inferred implementation root cause.

Test ID	Classification	Brief cause
API-001	`SUT_BUG`	A valid SAVE10 request returned `discount_amount=-4500000` and `final_amount=5000000`, contradicting the reviewed 10% formula.
API-002	`SUT_BUG`	BIGBUY was rejected when `total_amount` exactly equaled its allowed minimum of 500000; FR-09 defines the threshold as inclusive.
API-003	`SUT_BUG`	A total exactly equal to SAVE10's minimum was rejected instead of producing the specified calculation.
API-068	`SUT_BUG`	A `text/plain` body caused an unhandled destructuring error and HTTP 500 instead of a safe client-error response.
API-016	`SUT_BUG`	The valid reviewed SAVE10 baseline returned the same invalid negative percent discount.
API-022	`SUT_BUG`	BIGBUY at its exact minimum was rejected, violating the inclusive boundary.
API-031	`SUT_BUG`	SAVE10 at its exact minimum was rejected, violating the inclusive boundary.
API-033	`SUT_BUG`	A zero total was rejected for a coupon whose minimum is zero, violating the inclusive boundary.
API-051	`SUT_BUG`	The 10% calculation for total 1 returned `-9` rather than `0.1`.
API-052	`SUT_BUG`	An eligible 10% coupon returned the same invalid negative percent discount.
API-071	`SUT_BUG`	VIP100 at its exact minimum was rejected, violating the inclusive boundary.
API-073	`SUT_BUG`	A 1% coupon returned a zero discount for total 500000 instead of 5000.
API-074	`SUT_BUG`	A 100% coupon returned `-49500000` instead of 500000 and increased the final amount instead of reducing it to zero.

API-060	`SUT_BUG`	The authenticated valid baseline returned the invalid negative 10% calculation.
API-061	`SUT_BUG`	The operation returned a successful calculation with no JWT, contrary to FR-09 C4 and SEC-02.
API-062	`SUT_BUG`	The operation returned a successful calculation for a malformed JWT.
API-063	`SUT_BUG`	The operation returned a successful calculation for a JWT with an invalid signature.
API-064	`SUT_BUG`	The operation returned a successful calculation for an expired JWT.
API-067	`TEST_DEFECT` — **resolved**	The old assertion incorrectly treated any successful calculation as SQL/query bypass. Black-box execution can verify only observable effects such as database diagnostics; source inspection confirms bound parameters for both coupon and usage queries, the endpoint contains no write statement, the targeted rerun passed, and controlled database state was byte-for-byte unchanged before/after. This is not an SUT bug candidate.
API-072	`TEST_DEFECT` — **unsupported requirement assumption**	API specification § 5.1 explicitly supplies body `user_id`; FR-09 C4 requires a valid JWT and C5 limits uses by "the user," but neither defines JWT-subject/body binding. The reviewed context explicitly prohibits asserting that relationship. The SUT reads body `user_id` and does not derive it from the JWT for this route. Therefore the observed 200 response is valid under the documented body-driven behavior for this particular question, while the separate failure to enforce JWT remains covered by `API-061` – `API-064`. The human case was corrected to exploratory and removed from bug candidacy.

Six cases had no automated assertion but produced evidence that conflicts with an explicit reviewed requirement or mathematical formula:

Test ID	Classification	Brief cause
API-013	`SUT_BUG`	A request without an Authorization header received a successful discounted calculation despite the valid-JWT requirement.
API-014	`SUT_BUG`	A request with a known-invalid JWT received a successful discounted calculation.
API-020	`SUT_BUG`	The domain variant without a JWT was accepted and discounted.
API-021	`SUT_BUG`	The domain variant with an invalid JWT was accepted and discounted.
API-035	`SUT_BUG`	The manually observed percent result was `-4500005` / `5000005.5`, not the documented 10% / final-amount formulas.
API-047	`SUT_BUG`	The manually observed percent result was `-2700009` / `3000010`, not the documented formulas; precision policy cannot explain the magnitude or sign.

The 50 explicit manual-oracle cases are finally adjudicated in the `Final Manual and Execution-Only Review Resolution`: 15 `PASS`, 6 `FAIL`, 2 `BLOCKED`, and 27 `exploratory`. `API-067` remains completed by implementation inspection and the targeted before/after database comparison; `API-072` remains an exploratory observation because its former identity-binding oracle was unsupported.

No case is classified as `SETUP_DEFECT`: all 74 fixture states were reset and verified by the authenticated local controller, every intended ID ran, and Newman recorded no request, pre-request-script, or test-script errors.

Bug candidates and recommended next actions

- Treat the 24 `SUT_BUG` case observations above as bug candidates, not 24 proven unique bugs. Preserve the individual Test IDs when creating defects so each failing or suspicious observation remains traceable.
- Correct and retest percent calculation against `API-001`, `API-016`, `API-051`, `API-073`, and `API-074` first; these span ordinary, small-value, 1%, and 100% inputs.
- Correct and retest inclusive minimum eligibility with `API-002`, `API-003`, `API-022`, `API-031`, `API-033`, and `API-071`.
- Enforce JWT validation before coupon evaluation, then rerun `API-013`, `API-014`, `API-020`, `API-021`, and `API-061` – `API-064`.
- Return a controlled 4xx response for unsupported body media types and rerun `API-068`; retain `API-069` as the zero-byte-body robustness check.
- `API-067` is resolved: retain the corrected diagnostic assertion and the implementation/database evidence; do not report the historical failed assertion as an SUT bug.
- `API-072` is resolved as an unsupported requirement assumption: do not file a bug from its historical failure. The reviewed row and collection now preserve it only as an exploratory identity-scoping observation. A future binding requirement would need an authoritative specification change before becoming an automated oracle.
- Retain the final disposition of all 50 explicit manual checks in the `Final Manual and Execution-Only Review Resolution`; do not promote exploratory observations or blocked evidence gaps to SUT defects.

Targeted `API-067` rerun and implementation evidence

Metric	Result
Artifact	`reports/pool-b/api-067-rerun.json`
Execution window	2026-08-22 18:42:03.357 to 18:42:03.465 (GMT+7)
Scope	Exactly one request: `API-067`
Requests / tests / assertions	1 / 1 / 1
Outcome	`PASS`; 200 OK; 1 assertion passed, 0 failed; no request or script errors
Corrected black-box oracle	Response exposes no recognizable database query diagnostics; it does not require rejection or prohibit an ordinary calculation solely because `user_id` is a string
Implementation evidence	The coupon lookup and `coupon_usage` count lookup use `?` placeholders with separate bound parameter arrays. No client input is concatenated into either SQL statement. The apply-coupon route performs only reads.
State evidence	Controlled coupons, coupon usage, controlled users, and SQLite sequence snapshot were unchanged after the request; the pre-run SUT database snapshot was restored afterward

The original `API-067` 200 response could not prove or disprove parameterization by itself. The corrected automated check plus implementation and database-state evidence establish that the literal value did not alter query structure or state. Only this affected case was rerun.

Coverage and reconciliation notes

- All 74 intended collection leaf requests were observed; there are no blocked, skipped, filtered, or not-executed cases.
- Newman `run.stats` reports 148 requests because the runtime performs one fixture-reset helper request for each of the 74 SUT requests. The embedded collection still contains 74 reviewed leaf requests.
- The detailed execution array repeats each SUT execution object twice with the same immutable item ID and `httpRequestId`. Correlating those duplicate representations yields 74 logical executions, 27 assertions, and 20 assertion failures, matching Newman statistics; no duplicate error was counted twice.
- HTTP 4xx/5xx responses were not classified as failures merely because of status. `API-068` fails because its explicit assertion requires a status below 500; cases without a fixed HTTP oracle retain execution-only `PASS` and receive a final `exploratory` verdict.

- The original full-run totals remain 27 assertions (7 passed, 20 failed). The isolated corrected `API-067` rerun contributes a separate 1 passed assertion and does not rewrite the historical artifact. `API-072` was not rerun; its historical assertion failure is retained as evidence of the withdrawn test oracle, not as a product failure.

Pool C: Admin Order Management

C.1. Introduction

Pool C covers **FR-18 – Admin Order Management**. This report selects `PUT /api/admin/orders/:id/status`, which allows an Admin user to update an order to one of the supported states: `pending`, `confirmed`, `shipping`, `delivered`, or `canceled`.

C.2. Contractual Testing Analysis

The selected endpoint is documented as `PUT /api/admin/orders/:id/status`. It is part of the Admin API, requires a valid Bearer JWT whose token carries `role = 'admin'`, and accepts a JSON representation containing the requested order `status`:

Input	Documented representation	Reviewed constraint
`id`	Path placeholder in `/api/admin/orders/:id/status`	Identifies the order being updated; its type, syntax, range, canonical form, and nonexistent-order behavior are not specified
`Authorization`	`Bearer <token>` header	The JWT must be valid and contain `role = 'admin'`; token-validation details and failure responses are not specified
`status`	JSON string in the request body	Supported vocabulary is `pending`, `confirmed`, `shipping`, `delivered`, and `canceled`; every change must follow FR-10

The reviewed contract model contains rules `CR-001` – `CR-013`. These cover the method and route, structural presence of the path identifier, Bearer representation, JWT validity, Admin-role authorization, JSON body representation, string-valued `status`, the five-value vocabulary, lifecycle constraints, final-state guards, additional or duplicate body members, semantic success, and the limited invalid-transition error contract.

FR-10's diagram is treated as authoritative and exhaustive for non-self transitions. The five diagrammed transitions may succeed, while every omitted non-self transition is invalid. In particular, `shipping → canceled` is invalid: FR-10 line 161 does not authorize that edge, and FR-18 requires Admin changes to follow FR-10. Same-state requests remain unspecified and are not assigned a success or rejection oracle.

The endpoint has no documented success status, error status, response media type, response headers, response body schema, or exact message. FR-10 requires an invalid transition to return an error with an appropriate message, but does not define its wire format or wording. The sources also do not formally define body/field requiredness, nullability, coercion, malformed-body handling, additional properties, duplicate keys, or exact `Content-Type` behavior. Contract cases for those gaps are therefore exploratory or semantic-only and do not invent HTTP codes or schemas. The complete analysis is stored in `review/pool-c/reports/contract-report.md`.

C.3. Domain Testing Analysis

Domain analysis uses an existing order in `pending`, a valid Admin JWT, and `status = "confirmed"` as the valid baseline. The concrete order ID and token are controlled fixtures rather than specification constants. `Content-Type: application/json` is used as a representation assumption because the specification labels the body as JSON, but strict media-type behavior is not inferred.

The reviewed model defines 28 equivalence partitions (`DP-001` – `DP-028`) across the path identifier, authorization header, `status` value, and JSON body:

- `id` partitions cover an existing compatible order, an existing order whose source state makes the transition invalid, a nonexistent identifier, an omitted path segment, alternate lexical forms, and the literal path text `null`;

- authorization partitions cover a valid Admin JWT, an omitted or empty header, a wrong scheme or malformed representation, an invalid JWT, and a valid non-Admin JWT;
- `status` partitions cover all five documented values, an unknown value, an empty string, case or whitespace variants, `null`, non-string JSON types, and an omitted property;
- body partitions cover an empty or absent body, malformed JSON, a non-object top-level value, additional properties, and duplicate `status` properties.

The semantic result of a destination value depends on the current state of the order. The source-state fixture must therefore be controlled whenever a partition exercises transition validity. The five supported non-self transitions are `pending → confirmed`, `confirmed → shipping`, `shipping → delivered`, `pending → canceled`, and `confirmed → canceled`. The seven reviewed omitted transitions are invalid, and both `delivered` and `canceled` are final states. Same-state behavior remains outside the approved oracle.

No supported boundary-value IDs exist. The specification supplies no numeric, length, count, time, or other ordered limit for `id`, the Authorization header, `status`, or the JSON body. The five status strings form an enumeration, while lifecycle ordering belongs to state-transition testing; neither supports invented `DB-*` boundary cases.

The model preserves unresolved behavior for identifier syntax and existence, case/whitespace normalization, strict type enforcement, requiredness, nullability, coercion, duplicate/additional properties, malformed or non-object JSON, media types, exact response details, and JWT mechanics. `DP-004` is explicitly retained as an operation-level routing limitation because omitting the `id` path segment normally addresses a different route rather than supplying an empty value to this operation. The complete analysis is stored in `review/pool-c/reports/domain-report.md`.

C.4. State Transition Testing Analysis

State-transition testing is **applicable** because this endpoint mutates the order lifecycle. The reviewed state model contains five states:

State	Role in the lifecycle
`pending`	Initial state; may move to `confirmed` or `canceled`
`confirmed`	Intermediate state; may move to `shipping` or `canceled`
`shipping`	Intermediate state; may move only to `delivered`
`delivered`	Final state; cannot move to another state
`canceled`	Final state; cannot move to another state

The model defines five valid transitions (`TR-001` – `TR-005`): `pending → confirmed`, `confirmed → shipping`, `shipping → delivered`, `pending → canceled`, and `confirmed → canceled`. It also defines eight explicit final-state violations (`TR-006` – `TR-013`), covering each non-self request from `delivered` and `canceled` to one of the other four states.

Human review preserved `PR-001` – `PR-007` and classified every omitted non-self edge from a non-final source as invalid: `pending → shipping`, `pending → delivered`, `confirmed → pending`, `confirmed → delivered`, `shipping → pending`, `shipping → confirmed`, and `shipping → canceled`. For all invalid transitions, the supported oracle is an error with an appropriate message; the status, response schema, wording, and post-error persistence details remain unspecified.

Each transition case requires an existing order in the named source state and a valid Admin JWT. Authentication and authorization are endpoint guards rather than lifecycle states, and order creation or fixture preparation is outside this single-endpoint model. Same-state requests, idempotency, concurrent-update ordering, version checks, atomicity, rollback behavior, and nonexistent-order behavior remain unspecified. No same-state STATE candidates were generated. The complete analysis is stored in `review/pool-c/reports/state-report.md`.

C.5. Security Testing Analysis

All supplied requirements `SEC-01` – `SEC-07` were evaluated for this endpoint:

Requirement	Applicability and treatment

`SEC-01`	Not applicable: the endpoint has no password input or documented password-storage effect.
`SEC-02`	Applicable: this privileged mutation requires a valid JWT. Coverage includes a valid control, a missing header, and malformed, forged, expired, or otherwise invalid JWT fixture classes.
`SEC-03`	Applicable: an Admin API must verify `role = 'admin'`; a valid non-Admin token must not authorize the mutation.
`SEC-04`	Not applicable at endpoint level: no response-reflection or UI-rendering behavior is specified for `id` or `status`.
`SEC-05`	Applicable: database lookup and mutation use client-controlled `id` and `status`; inert query-shaped values must remain data and must not broaden or redirect updates.
`SEC-06`	Not applicable: this is not a profile-update API and `role` is not a documented body field.
`SEC-07`	Not applicable: the endpoint does not handle password-reset OTPs.

The reviewed security model contains seven scenarios (`SS-001` – `SS-007`). `SS-001` is the valid JWT/Admin-role control. `SS-002` checks a missing Authorization header, `SS-003` covers environment-classified invalid JWT fixtures, and `SS-004` verifies that a valid non-Admin JWT cannot mutate the order. These negative cases assert denial and absence of mutation without inventing an HTTP status, response body, or validation precedence.

`SS-005` and `SS-006` use inert SQL-control-shaped values in path `id` and body `status`. The values must be treated as data: they must not execute query instructions, broaden the affected row set, bypass lifecycle constraints, or modify targeted or unrelated orders unexpectedly. `SS-007` is the ordinary valid-update control and verifies that only the identified order changes.

Black-box behavior can expose unsafe effects but cannot conclusively prove implementation-level query parameterization; source inspection or database instrumentation may still be required. The requirements do not define JWT algorithms, issuer, audience, expiry mechanics, claim encoding, authentication/validation precedence, TLS, rate limits, throttling, replay controls, lockout, or audit logging for this endpoint. The complete analysis is stored in `review/pool-c/reports/security-report.md`.

C.6. AI-Generated Test Cases

The final reviewed test cases for this pool are stored in `test-cases/c-order-management.csv`. The audited AI-generated subset is stored in `review/pool-c/candidate-api-tests.csv`.

The AI-generated cases were derived from the reviewed analyses in Sections C.2–C.5. After semantic deduplication, the 79 retained AI cases were combined with the six human-authored cases in Section C.7:

Testing Type	AI-generated	Human-added	Final total
Contractual Testing	23	1	24
Domain Testing	26	2	28
State Transition Testing	20	2	22
Security Testing	10	1	11
Total	**79**	**6**	**85**

The AI subset uses sequential IDs `API-001` – `API-079`; the final suite appends human-authored `API-080` – `API-085` and remains sequential through `API-085`. Every case targets `PUT /api/admin/orders/:id/status`. The final CSV uses the established nine traceability fields. The audited candidate CSV preserves those same fields for the AI subset and adds `Audit Result` and `Audit Reason`; every retained AI case is marked `VALID` with case-level reasoning.

AI traceability is complete for `CR-001` – `CR-013`, `DP-001` – `DP-028`, `TR-001` – `TR-013`, `PR-001` – `PR-007`, and `SS-001` – `SS-007`, including applicable `SEC-02`, `SEC-03`, and `SEC-05`. No `DB-*` items exist because the reviewed domain model found no supported ordered boundaries, and no same-state STATE case was generated.

Same-category semantic deduplication removed two DOMAIN records by merging former `API-024`, `API-030`, and `API-036` into retained `API-024`. All three used the same valid pending-to-confirmed request and semantic outcome. The retained case preserves `DP-001`, `DP-007`, and `DP-013` plus provisional specialist IDs `DOMAIN-P001`, `DOMAIN-P007`, and `DOMAIN-P013`.

Final validation confirmed 85 sequential unique IDs, the exact nine-column final schema, endpoint/category consistency, non-empty required content, unchanged AI-generated rows `API-001` – `API-079`, and clear ordered steps for the six human workflows. Unspecified behavior remains unresolved: no AI case invents numeric HTTP statuses, response schemas, exact messages, identifier rules, same-state semantics, or unsupported JWT mechanics. Pool C Postman generation and Newman execution were subsequently completed; Section C.8 analyzes the resulting collection and Newman JSON/HTML evidence.

C.7. Human Cases

ID	Category	Test case	Expected result	Notes
API-080	CONTRACT	Send otherwise valid JSON text for a `pending → confirmed` update using `Content-Type: text/plain`. Then retry the same order with `Content-Type: application/json`.	The first response is a safe `4xx`, not `5xx`, and does not change the order status. The second request succeeds and changes `pending → confirmed`.	The AI tested missing Content-Type, malformed JSON, and missing bodies, but not valid JSON sent with an unsupported content type. The accepted `4xx` class is a human/external HTTP expectation.
API-081	DOMAIN	For a pending order, send `status` as an object: `{"value":"confirmed"}`. Then retry with `status: "confirmed"`.	The first request returns a `4xx` error and does not change the order. The valid retry succeeds with `pending → confirmed`.	The AI used one representative non-string value. This case exercises a different JSON structural representation and verifies that rejection has no state side effect.
API-082	DOMAIN	For a pending order, send `status: " confirmed "` with leading and trailing spaces. Then retry with exact `status: "confirmed"`.	The whitespace-modified value is rejected with a `4xx` and leaves the order pending. The exact-value retry succeeds.	The AI tested case variation such as a differently cased state, but not surrounding whitespace. The documented lifecycle uses exact state names.
API-083	STATE	Start with an order in `shipping`. First request `shipping → canceled`, then request `shipping → delivered` on the same order.	The first request is rejected with `4xx` and must not change the state. The second request succeeds, proving the invalid transition left the order in `shipping`.	The AI tests invalid transitions individually, but does not verify with a follow-up transition that a rejected request leaves the source state unchanged.
API-084	STATE	On one order, execute the full sequence `pending → confirmed → shipping → delivered`, then attempt `delivered → canceled`.	The first three updates succeed in order. The final request is rejected with `4xx` because `delivered` is a final state.	The AI tests the transitions separately. This human case verifies the complete lifecycle on one persistent order and the final-state rule at the end of the workflow.
API-085	SECURITY	For a pending order, first send `pending → confirmed` using a valid JWT for a non-Admin user. Then repeat the same update using a valid Admin JWT.	The non-Admin request is rejected with a `4xx` and must not change the order. The Admin retry succeeds with `pending → confirmed`.	The AI checks that a non-Admin JWT is denied, but this case additionally proves that the unauthorized mutation did not change persistent order state.

The AI missed these cases mainly because the generated tests focused on individual contract partitions, input classes, and state transitions. This gives broad coverage, but it does not always verify what happens to persistent state after a rejected request.

The AI included representative cases for malformed JSON, missing fields, non-string values, and status casing, but did not cover valid JSON sent with an unsupported content type, object-valued status, or surrounding whitespace.

The human cases therefore add cross-request verification. They use a second request to prove that invalid input or unauthorized access did not silently change the order, and they exercise the complete order lifecycle on one order. All six cases can be implemented as ordered Postman requests and executed automatically with Newman.

C.8. Newman Execution Analysis

Scope and evidence

This analysis uses the unchanged Newman JSON artifact `reports/pool-c/pool-c.json`, the generated collection `postman/pool-c-order-management.postman_collection.json`, and the 85 reviewed logical cases in `test-cases/c-order-management.csv`. No test was modified and Newman was not rerun.

Execution status and defect triage are separate. `FAIL_ASSERTION` records a failed reviewed automated oracle; the later triage table determines whether that evidence indicates an SUT, test, or setup defect. A `PASS` whose only assertion checks the required `X-Student-Id` header confirms execution only, not the exploratory behavior described by the reviewed case.

Run summary

Metric	Normalized result
Collection	`Pool C — Admin Order Management — 85 Reviewed Cases`
Execution window	2026-08-22 23:26:00.279 to 23:26:08.331 (GMT+7)
Duration	8.052 seconds
Reviewed requests / logical cases	93 / 85; all 93 embedded leaf-item IDs executed and all `API-001` – `API-085` were observed
Request reconciliation	245 total = 93 reviewed SUT requests + 85 fixture-reset helpers + 67 post-response state-oracle helpers
Why 93 requests map to 85 cases	`API-080`, `API-081`, `API-082`, `API-083`, and `API-085` contain 2 steps each; `API-084` contains 4 steps. Those six flows add 8 leaf requests to the 85 logical IDs.
Newman tests / scripts	93 tests; 93 test scripts; 93 pre-request scripts; 186 total script executions; no script failures
Assertions	298 total: 290 passed, 8 failed, 0 pending/skipped
Request / pre-request-script / test-script errors	0 / 0 / 0
Logical execution statuses	`PASS`: 78; `FAIL_ASSERTION`: 7; `REQUEST_ERROR`: 0; `RUNTIME_ERROR`: 0; `BLOCKED_NOT_EXECUTED`: 0; `NOT_EXECUTED`: 0
Logical cases with failed automated assertions	7: `API-006`, `API-008`, `API-034`, `API-062`, `API-076`, `API-080`, `API-085`
Failed-assertion triage	`SUT_BUG`: 8 assertions; `TEST_DEFECT`: 0; `SETUP_DEFECT`: 0; failed assertions requiring classification-only manual review: 0

Per-logical-case outcomes

For items with a state oracle, the JSON detailed-execution row retains the final `GET /state/<Test-ID>` response rather than a separate SUT response. Those rows therefore report the helper's HTTP 200 and mark the SUT status unavailable; failure messages that captured an SUT code (`API-080` and `API-085`) retain that original code. Correlating identical rows by embedded immutable item ID and `httpRequestId` avoids counting helper-induced duplicates as repeated logical executions.

Test ID	Execution Status	Request / Flow Step	HTTP Status	Assertion Result	Failure / Error Message	Manual Oracle Required	Execution Notes
API-001	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 3/3	N/A	NO	Item-ID correlation; semantic state oracle passed.
API-002	`PASS`	1 reviewed POST request	404 Not Found	PASS — header/execution only	N/A	YES — exploratory alternate-method behavior	SUT response preserved.
API-003	`PASS`	1 reviewed request	404 Not Found	PASS — header/execution only	N/A	YES — exploratory altered-route behavior	SUT response preserved.
API-004	`PASS`	1 reviewed request	404 Not Found	PASS — header/execution only	N/A	YES — exploratory nonexistent-ID behavior	SUT response preserved.
API-005	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 4/4	N/A	NO	Denial and unchanged-state oracles passed.
API-006	`FAIL_ASSERTION`	1 reviewed request	SUT unavailable; state helper 200	FAIL — 1/4	`target order state is pending`: expected `confirmed` to equal `pending`	NO	State helper found target `300006` changed to `confirmed`.
API-007	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 4/4	N/A	NO	Invalid-JWT denial and state oracles passed.
API-008	`FAIL_ASSERTION`	1 reviewed request	SUT unavailable; state helper 200	FAIL — 1/4	`target order state is pending`: expected `confirmed` to equal `pending`	NO	Valid `role=user` token changed target `300008`.
API-009	`PASS`	1 reviewed request	400 Bad Request	PASS — header/execution only	N/A	YES — exploratory malformed-JSON behavior	SUT response preserved.

API-010	PASS	1 reviewed request	400 Bad Request	PASS — header/execution only	N/A	YES — exploratory non-object-body behavior	SUT response preserved.
API-011	PASS	1 reviewed request	500 Internal Server Error	PASS — header/execution only	N/A	YES — exploratory omitted-body behavior	Execution-only pass; retained as a manual-review robustness observation.
API-012	PASS	1 reviewed request	400 Bad Request	PASS — header/execution only	N/A	YES — exploratory omitted-status behavior	SUT response preserved.
API-013	PASS	1 reviewed request	400 Bad Request	PASS — header/execution only	N/A	YES — exploratory null-status behavior	SUT response preserved.
API-014	PASS	1 reviewed request	400 Bad Request	PASS — header/execution only	N/A	YES — exploratory non-string-status behavior	SUT response preserved.
API-015	PASS	1 reviewed request	SUT unavailable; state helper 200	PASS — 3/3	N/A	NO	Unsupported-status behavior oracle passed.
API-016	PASS	1 reviewed request	200 OK	PASS — header/execution only	N/A	YES — exploratory additional-property behavior	SUT response preserved.
API-017	PASS	1 reviewed request	200 OK	PASS — header/execution only	N/A	YES — exploratory duplicate-key behavior	SUT response preserved.
API-018	PASS	1 reviewed request	SUT unavailable; state helper 200	PASS — 3/3	N/A	NO	Permitted transition oracle passed.
API-019	PASS	1 reviewed request	SUT unavailable; state helper 200	PASS — 5/5	N/A	NO	Invalid transition, message, and state oracles passed.
API-020	PASS	1 reviewed request	SUT unavailable; state helper 200	PASS — 5/5	N/A	NO	Delivered final-state oracle passed.

API-021	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 5/5	N/A	NO	Canceled final-state representative passed.
API-022	`PASS`	1 reviewed request	400 Bad Request	PASS — header/execution only	N/A	YES — exploratory same-state behavior	SUT response preserved.
API-023	`PASS`	1 reviewed request	200 OK	PASS — header/execution only	N/A	YES — exploratory omitted-Content-Type behavior	SUT response preserved.
API-024	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 3/3	N/A	NO	Valid baseline state oracle passed.
API-025	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 5/5	N/A	NO	Invalid source-state oracle passed.
API-026	`PASS`	1 reviewed request	404 Not Found	PASS — header/execution only	N/A	YES — exploratory nonexistent-ID behavior	SUT response preserved.
API-027	`PASS`	1 reviewed request	404 Not Found	PASS — header/execution only	N/A	YES — operation-level omitted-ID routing	SUT response preserved.
API-028	`PASS`	1 reviewed request	404 Not Found	PASS — header/execution only	N/A	YES — exploratory text-ID behavior	SUT response preserved.
API-029	`PASS`	1 reviewed request	404 Not Found	PASS — header/execution only	N/A	YES — exploratory literal-`null` ID behavior	SUT response preserved.
API-030	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 4/4	N/A	NO	Missing-authorization denial and state oracles passed.
API-031	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 4/4	N/A	NO	Empty-Bearer denial and state oracles passed.
API-032	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 4/4	N/A	NO	Wrong-scheme denial and state oracles passed.

API-033	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 4/4	N/A	NO	Invalid-JWT denial and state oracles passed.
API-034	`FAIL_ASSERTION`	1 reviewed request	SUT unavailable; state helper 200	FAIL — 1/4	`target order state is pending`: expected `confirmed` to equal `pending`	NO	Valid `role=user` token changed target `300034`.
API-035	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 3/3	N/A	NO	Confirmed-to-shipping oracle passed.
API-036	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 3/3	N/A	NO	Shipping-to-delivered oracle passed.
API-037	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 3/3	N/A	NO	Pending-to-canceled oracle passed.
API-038	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 5/5	N/A	NO	Confirmed-to-pending rejection passed.
API-039	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 3/3	N/A	NO	Unsupported vocabulary oracle passed.
API-040	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 3/3	N/A	NO	Empty-string status oracle passed.
API-041	`PASS`	1 reviewed request	400 Bad Request	PASS — header/execution only	N/A	YES — exploratory case-normalization behavior	SUT response preserved.
API-042	`PASS`	1 reviewed request	400 Bad Request	PASS — header/execution only	N/A	YES — exploratory nullability behavior	SUT response preserved.
API-043	`PASS`	1 reviewed request	400 Bad Request	PASS — header/execution only	N/A	YES — exploratory type/coercion behavior	SUT response preserved.

API-044	PASS	1 reviewed request	400 Bad Request	PASS — header/execution only	N/A	YES — exploratory omitted-property behavior	SUT response preserved.
API-045	PASS	1 reviewed request	400 Bad Request	PASS — header/execution only	N/A	YES — exploratory absent-body behavior	SUT response preserved.
API-046	PASS	1 reviewed request	400 Bad Request	PASS — header/execution only	N/A	YES — exploratory malformed-JSON behavior	SUT response preserved.
API-047	PASS	1 reviewed request	400 Bad Request	PASS — header/execution only	N/A	YES — exploratory non-object-body behavior	SUT response preserved.
API-048	PASS	1 reviewed request	200 OK	PASS — header/execution only	N/A	YES — exploratory additional-property behavior	SUT response preserved.
API-049	PASS	1 reviewed request	400 Bad Request	PASS — header/execution only	N/A	YES — exploratory duplicate-key behavior	SUT response preserved.
API-050	PASS	1 reviewed request	SUT unavailable; state helper 200	PASS — 3/3	N/A	NO	Pending-to-confirmed oracle passed.
API-051	PASS	1 reviewed request	SUT unavailable; state helper 200	PASS — 3/3	N/A	NO	Confirmed-to-shipping oracle passed.
API-052	PASS	1 reviewed request	SUT unavailable; state helper 200	PASS — 3/3	N/A	NO	Shipping-to-delivered oracle passed.
API-053	PASS	1 reviewed request	SUT unavailable; state helper 200	PASS — 3/3	N/A	NO	Pending-to-canceled oracle passed.
API-054	PASS	1 reviewed request	SUT unavailable; state helper 200	PASS — 3/3	N/A	NO	Confirmed-to-canceled oracle passed.
API-055	PASS	1 reviewed request	SUT unavailable; state helper 200	PASS — 5/5	N/A	NO	Delivered-to-pending rejection passed.

API-056	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 5/5	N/A	NO	Delivered-to-confirmed rejection passed.
API-057	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 5/5	N/A	NO	Delivered-to-shipping rejection passed.
API-058	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 5/5	N/A	NO	Delivered-to-canceled rejection passed.
API-059	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 5/5	N/A	NO	Canceled-to-pending rejection passed.
API-060	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 5/5	N/A	NO	Canceled-to-confirmed rejection passed.
API-061	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 5/5	N/A	NO	Canceled-to-shipping rejection passed.
API-062	`FAIL_ASSERTION`	1 reviewed request	SUT unavailable; state helper 200	FAIL — 1/5	`target order state is canceled`: expected `delivered` to equal `canceled`	NO	State helper found target `300062` changed to `delivered`.
API-063	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 5/5	N/A	NO	Pending-to-shipping rejection passed.
API-064	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 5/5	N/A	NO	Pending-to-delivered rejection passed.
API-065	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 5/5	N/A	NO	Confirmed-to-pending rejection passed.
API-066	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 5/5	N/A	NO	Confirmed-to-delivered rejection passed.

API-067	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 5/5	N/A	NO	Shipping-to-pending rejection passed.
API-068	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 5/5	N/A	NO	Shipping-to-confirmed rejection passed.
API-069	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 5/5	N/A	NO	Shipping-to-canceled rejection passed.
API-070	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 3/3	N/A	NO	Valid Admin security control passed.
API-071	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 4/4	N/A	NO	Missing-JWT denial and state oracles passed.
API-072	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 4/4	N/A	NO	Malformed-JWT denial and state oracles passed.
API-073	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 4/4	N/A	NO	Forged-JWT denial and state oracles passed.
API-074	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 4/4	N/A	NO	Expired-JWT denial and state oracles passed.
API-075	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 4/4	N/A	NO	Not-yet-valid JWT denial and state oracles passed.
API-076	`FAIL_ASSERTION`	1 reviewed request	SUT unavailable; state helper 200	FAIL — 1/4	`target order state is pending`: expected `confirmed` to equal `pending`	NO	Valid `role=user` token changed target `300076`.

API-077	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 5/5	N/A	NO — source inspection completed	Inert ID changed no controlled order; query uses a bound `?` parameter.
API-078	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 5/5	N/A	NO — source inspection completed	Inert status changed no order; read and update queries use bound parameters.
API-079	`PASS`	1 reviewed request	SUT unavailable; state helper 200	PASS — 5/5	N/A	NO — source inspection completed	Only the identified target changed; update uses bound parameters.
API-080	`FAIL_ASSERTION`	Step 1 text/plain; Step 2 JSON retry	Step 1: 500 captured by assertion; Step 2: SUT unavailable, state helper 200	FAIL — 1/7	`reviewed response class is 4xx`: expected 500 to be within 400..499	YES — confirm human/external 4xx robustness authority	Step 1 left state pending; Step 2 changed it to confirmed.
API-081	`PASS`	2-step ordered flow	SUT unavailable; state helpers 200	PASS — 7/7	N/A	NO	Object value rejected without mutation; string retry succeeded.
API-082	`PASS`	2-step ordered flow	SUT unavailable; state helpers 200	PASS — 7/7	N/A	NO	Whitespace value rejected without mutation; exact retry succeeded.
API-083	`PASS`	2-step ordered flow	SUT unavailable; state helpers 200	PASS — 7/7	N/A	NO	Invalid shipping-to-canceled left state intact; delivered retry succeeded.
API-084	`PASS`	4-step ordered flow	SUT unavailable; state helpers 200	PASS — 13/13	N/A	NO	Full lifecycle passed; final delivered-to-canceled attempt was rejected.

API-085	`FAIL_ASSERTION`	Step 1 non-Admin; Step 2 Admin retry	Step 1: 200 captured by assertion; Step 2: SUT unavailable, state helper 200	FAIL — 2/11	`reviewed response class is 4xx`: expected 200 within 400..499; `target order state is pending`: expected `confirmed` to equal `pending`	NO	Step 1 unauthorized mutation succeeded; Step 2 valid control also ended confirmed.

Failed-assertion triage

The table below classifies each of the eight assertion events once; duplicated representations in `run.executions` and `run.failures` are correlated rather than recounted.

Test ID / assertion	Classification	Preserved evidence and rationale
`API-006` — target remains pending	`SUT_BUG`	A `Basic <valid-admin-JWT>` header produced a mutation from pending to confirmed. The API contract requires `Bearer`; source inspection shows `authenticateToken` blindly takes the second whitespace-delimited token and never validates the scheme.
`API-008` — target remains pending	`SUT_BUG`	A valid fixture JWT with `role=user` changed the target to confirmed, contrary to FR-12 and SEC-03.
`API-034` — target remains pending	`SUT_BUG`	The independent DOMAIN case reproduced the same valid-non-Admin mutation with its own isolated target.
`API-062` — target remains canceled	`SUT_BUG`	The fixture began in final state canceled, but the state oracle observed delivered. Source inspection confirms an explicit erroneous `canceled → delivered` branch.
`API-076` — target remains pending	`SUT_BUG`	The SECURITY case independently reproduced the missing Admin-role enforcement.
`API-080` — response class is 4xx	`SUT_BUG` candidate under the reviewed human robustness oracle	The SUT returned 500 and an unhandled `TypeError` when `bodyParser.json()` left `req.body` undefined for `text/plain`; state remained pending and the JSON retry succeeded. Because the 4xx rule is labeled a human/external expectation rather than a formal FR/API response contract, its requirement authority should be confirmed before filing.
`API-085` step 1 — response class is 4xx	`SUT_BUG`	A valid non-Admin token received 200 instead of a denial class. This is direct HTTP evidence for the same missing role check.
`API-085` step 1 — target remains pending	`SUT_BUG`	The state helper independently confirmed the same unauthorized request changed the target from pending to confirmed while the unrelated sentinel remained unchanged.

No failed assertion is a `TEST_DEFECT` or `SETUP_DEFECT`. The reviewed oracles align with FR-10, FR-12, FR-18, SEC-03, or the explicit human case; the fixture runner generated distinct valid Admin and `role=user` JWTs, verified every reset, isolated exactly two order rows, and recorded no request or script errors. `API-080` retains a manual requirement-authority check, but its recorded 500 and source-level unhandled exception are genuine execution evidence rather than a setup failure.

Root-cause bug candidates

Candidate root cause	Test/assertion traceability	Evidence
Authorization scheme is not validated	`API-006`	`authenticateToken` splits `Authorization` and verifies element 1 without requiring the `Bearer` scheme, so `Basic <JWT>` is accepted.
Admin role is never enforced on `/api/admin/*`	`API-008`, `API-034`, `API-076`, `API-085` (both failed step-1 assertions)	The middleware verifies JWT validity but never checks `req.user.role`; the order-status route adds no role guard. Three independent single-request cases and one two-step flow reproduce it.
Canceled is incorrectly allowed to transition to delivered	`API-062`	The route contains an explicit `if (currentStatus === "canceled" && status === "delivered") isValidTransition = true`, contradicting the FR-10 final-state rule.
Undefined request body is destructured without a guard	`API-080`; related manual-review observation `API-011`	`const { status } = req.body` throws when the JSON parser does not populate `req.body`. `API-080` captures HTTP 500 for `text/plain`; exploratory `API-011` independently records HTTP 500 for an absent body.

These are four bug candidates, not eight separate defects. Any issue report should retain every listed Test ID and both `API-085` assertion messages so the consolidated root cause does not erase case-level evidence.

Manual-oracle and coverage notes

- Final review classifies all twenty-six execution-only characterization cases as `exploratory`: `API-002` – `API-004`, `API-009` – `API-014`, `API-016`, `API-017`, `API-022`, `API-023`, `API-026` – `API-029`, and `API-041` – `API-049`. Their only Newman assertion checks the required student header, and the specification supplies no behavior oracle.
- Final review classifies `API-080` as `FAIL` under its explicit reviewed human/external 4xx robustness oracle: the observed 500 and unhandled exception contradict that oracle, while captured state evidence proves step 1 did not mutate the order.
- Source inspection completed the non-black-box parameterization question for `API-077` – `API-079`: the lookup and update statements use `?` placeholders with separate parameter arrays, consistent with their passing whole-table state oracles.
- The detailed execution array contains 34 item IDs represented twice and 59 represented three times. This is explained exactly by the helper traffic: every logical ID has one reset, and 67 leaf requests perform a state query; eight continuation steps have a state query but no additional reset. The reconstructed 93 unique items and 298 unique assertions match `run.stats` exactly.
- HTTP 4xx/5xx values were not treated as failures by themselves. `API-011` is finalized as execution-only `exploratory` despite HTTP 500; `API-080` fails because its explicit reviewed assertion requires 4xx.

Final Manual and Execution-Only Review Resolution

All remaining manual-oracle and execution-only cases across Pools A, B, and C have been adjudicated without changing the historical Newman results. The authoritative per-case final-review verdicts and evidence limits are recorded in `reports/manual-execution-review-resolution.md`.

The 98 reviewed cases resolve to **15 PASS, 15 FAIL, 14 BLOCKED, and 54 exploratory**. A `BLOCKED` verdict means the required persistence, implementation, timing, expiry, or configuration evidence is unavailable; an `exploratory` verdict means the specification defines no pass/fail oracle for the observed behavior.